



SVEUČILIŠTE JOSIPA JURJA STROSSMAYERA U OSIJEKU

ODJEL ZA MATEMATIKU

Sveučilišni diplomski studij matematike
smjer: Matematika i računarstvo

OWASP Top 10: SQL Injection napadi i zaštita

SEMINARSKI RAD

Jurica Jurčević

Osijek, 2026.

Sadržaj

1	Uvod	1
2	Kako nastaje SQL injection	3
2.1	Podatak i naredba u istom nizu znakova	3
2.2	Tok podataka	3
3	Vrste napada	5
3.1	Napadi s izravnim odgovorom	5
3.1.1	UNION napad	5
3.1.2	Napad kroz poruku o pogrešci	5
3.2	Slijepi napadi	5
3.2.1	Bulov slijepi napad	6
3.2.2	Vremenski slijepi napad	6
3.3	Napad drugog reda	6
4	Demonstracija napada	7
4.1	Postavljanje	7
4.2	Napadi	7
4.3	Ista provjera protiv sigurne inačice	8
5	Zaštita	9
5.1	Pripremljeni upiti	9
5.2	Provjera ulaza	9
5.3	Najmanje ovlasti	9
5.4	Slojevi iznad aplikacije	10
6	Zaključak	11
	Literatura	13
	Sažetak	15
	Summary	17

1 | Uvod

SQL injection je napad u kojem se ulaz korisnika probije iz podatkovnog dijela upita u njegov naredbeni dio. Aplikacija očekuje ime ili broj, a napadač pošalje komad SQL koda koji baza posluša. Posljedica je čitanje tuđih podataka, zaobilaznje prijave ili, u težim slučajevima, potpuni nadzor nad serverom baze.

Napad je star koliko i web aplikacije s bazama, a i dalje je aktualan. OWASP Top 10 iz 2021. godine svrstava ga u kategoriju *A03: Injection*, koja je pala s prvog na treće mjesto, no i dalje pogađa velik dio testiranih aplikacija [1]. Razlog dugovječnosti je jednostavan. Spajanje niza znakova u tekst upita djeluje prirodno svakom programeru koji uči raditi s bazom, a posljedice tog spajanja nisu očite dok ih netko ne pokaže.

Ovaj rad opisuje kako napad radi, koje vrste postoje i kako se sprječava. Drugo poglavlje objašnjava mehanizam na razini jednog upita. Treće poglavlje razvrstava napade prema tome kako napadač dolazi do rezultata. Četvrto poglavlje sadrži demonstraciju u kontroliranom okruženju, s bazom koja se stvara u radnoj memoriji i nikad ne izlazi iz procesa. Peto poglavlje pokriva obranu, s naglaskom na pripremljene upite jer oni uklanjaju uzrok, a ne samo simptome. Svi primjeri koda i ispisi u radu proizvedeni su pokretanjem priloženih skripti.

2 | Kako nastaje SQL injection

2.1 Podatak i naredba u istom nizu znakova

Baza podataka prima naredbe kao tekst. Kad aplikacija gradi taj tekst lijepljenjem korisničkog ulaza, granica između onoga što je naredba i onoga što je podatak nestaje. Baza vidi samo gotov niz znakova i izvršava ga u cijelosti.

Pogledajmo tipičan upit za prijavu:

```
1 SELECT id, uloga FROM korisnici
2 WHERE korisnicko_ime = 'ana' AND lozinka_hash = '...';
```

Vrijednost 'ana' dolazi iz obrasca za prijavu. Ako je aplikacija sastavila upit ovako:

```
1 $upit = "... WHERE korisnicko_ime = '" . $ime . "' AND ...";
```

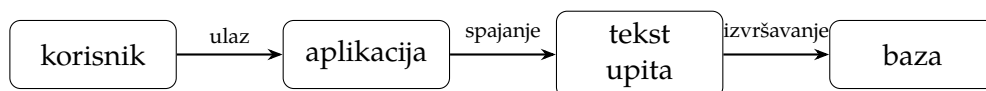
onda napadač kroz polje \$ime ne mora poslati ime. Može poslati admin' -- . Nakon lijepljenja baza dobiva:

```
1 SELECT id, uloga FROM korisnici
2 WHERE korisnicko_ime = 'admin' -- ' AND lozinka_hash = '...';
```

Dva znaka -- u SQL-u započinju komentar do kraja retka. Sve iza njih baza zanemaruje, uključujući provjeru lozinke. Upit sada vraća redak korisnika admin bez ijedne lozinke. Apostrof je zatvorio niz znakova koji je programer otvorio, a ostatak ulaza postao je dio naredbe.

2.2 Tok podataka

Slika 2.1 prikazuje put ulaza kroz ranjivu aplikaciju. Ulaz prolazi kroz preglednik i aplikacijski sloj bez provjere, spaja se u upit i stiže u bazu kao gotova naredba. Napad je moguć točno na koraku spajanja. Kad bi taj korak razdvajao tekst upita od vrijednosti, ulaz nikad ne bi mogao promijeniti značenje naredbe. Upravo to rade pripremljeni upiti iz Poglavlja 5.



Slika 2.1: Tok ulaza u ranjivoj aplikaciji. Nigdje se ne razdvaja naredba od podatka.

3 | Vrste napada

Napade dijelimo prema tome kako napadač dolazi do informacije. Podjela nije formalna, ali pomaže u razumijevanju obrane jer svaka vrsta traži drukčiji kanal odgovora.

3.1 Napadi s izravnim odgovorom

Kod ovih napada rezultat se vraća u istom odgovoru aplikacije. To je najbrži put jer napadač odmah vidi što je dobio.

3.1.1 UNION napad

Operator UNION spaja rezultat dvaju upita u jednu tablicu. Ako napadač zna broj i tip stupaca u izvornom upitu, može dodati vlastiti SELECT i pročitati bilo koju tablicu na koju korisnik baze ima pravo. Uvjet je da se broj stupaca poklapa.

```
1 ' UNION SELECT id, vlasnik, broj FROM kartice --
```

Ovakav ulaz u polju za pretragu proizvoda vraća sadržaj tablice kartice umjesto proizvoda. Demonstracija u Poglavlju 4 pokazuje točno ovaj slučaj.

3.1.2 Napad kroz poruku o pogrešci

Ako aplikacija korisniku prikazuje poruke baze o pogreškama, napadač može navesti bazu da vrati podatak unutar same poruke. Primjerice, pretvaranje niza znakova u broj izazove pogrešku koja sadrži taj niz. Ova vrsta danas je rjeđa jer produkcijske aplikacije obično skrivaju detalje pogrešaka, no na testnim i loše postavljenim sustavima i dalje se pojavljuje.

3.2 Slijepi napadi

Kad aplikacija ne vraća ni rezultat upita ni poruke o pogreškama, napadač i dalje može izvlačiti podatke. Postavlja pitanja na koja baza odgovara s da ili ne, i zaključuje po ponašanju aplikacije. Ovo je sporije, jer se svaki bit izvlači zasebno, ali radi.

3.2.1 Bulov slijepi napad

Napadač ubaci uvjet i promatra razliku u odgovoru. Ako se stranica prikaže normalno za $AND\ 1=1$ a drukčije za $AND\ 1=2$, aplikacija propušta jedan bit informacije po zahtjevu. Nizanjem takvih uvjeta čita se, znak po znak, ime baze, imena tablica i njihov sadržaj.

3.2.2 Vremenski slijepi napad

Kad se odgovor uopće ne mijenja, ostaje vrijeme. Napadač ubaci naredbu koja usporava bazu ako je uvjet istinit, primjerice funkciju koja čeka nekoliko sekundi. Duljina odgovora tada nosi informaciju. Ova varijanta radi čak i kad aplikacija baš ništa ne vraća napadaču, pa je i najsporija.

3.3 Napad drugog reda

Kod napada drugog reda zlonamjerni ulaz se prvo pohrani, naizgled bezopasno, a tek se poslije upotrijebi u nekom drugom upitu. Aplikacija je oprezna pri unosu, ali podatak iz vlastite baze tretira kao pouzdan i lijepi ga u novi upit. Ovo je podmuklo jer provjera na ulazu ne pomaže. Uzrok je isti kao i uvijek, spajanje niza znakova umjesto razdvajanja naredbe od podatka.

4 | Demonstracija napada

4.1 Postavljanje

Demonstracija koristi `sqlite3` iz standardne biblioteke jezika Python i bazu u radnoj memoriji. Ništa se ne sprema na disk i ništa ne izlazi iz procesa. Baza ima tablicu `korisnici` s tri računa i tablicu `kartice` s osjetljivim podacima, kako bi se vidjelo do čega napad može doći.

Skripta uspoređuje dvije inačice funkcije za prijavu. Prva lijepi ulaz u tekst upita, druga koristi pripremljeni upit. Cijeli kod je u prilogu `demo_sql.py`, a ovdje su ključni dijelovi.

```
1 def prijava_ranjiva(veza, ime, lozinka):
2     upit = (
3         "SELECT id, korisnicko_ime, uloga FROM korisnici "
4         f"WHERE korisnicko_ime = '{ime}' AND lozinka_hash = '{lozinka}'"
5     )
6     return veza.execute(upit).fetchall()
```

Listing 4.1: Ranjiva funkcija za prijavu

4.2 Napadi

Prvi napad zaobilazi provjeru lozinke. Kao korisničko ime pošalje se `admin' --`, a lozinka je nevažna. Ispis skripte pokazuje upit koji baza stvarno vidi i redak koji je vraćen:

```
1 upit koji baza vidi:
2  SELECT id, korisnicko_ime, uloga FROM korisnici
3  WHERE korisnicko_ime = 'admin' -- ' AND lozinka_hash = 'bilo sto'
4 rezultat (1 redaka):
5  (3, 'admin', 'administrator')
```

Prijava je uspjela bez ispravne lozinke. Drugi napad koristi uvjet koji je uvijek istinit, `' OR '1'='1`, u oba polja i vraća sve korisnike odjednom:

```
1 rezultat (3 redaka):
2  (1, 'ana', 'korisnik')
```

```
3 (2, 'marko', 'korisnik')
4 (3, 'admin', 'administrator')
```

Treći napad je UNION. Kao korisničko ime pošalje se sljedeći niz:

```
1 ' UNION SELECT id, vlasnik, broj FROM kartice --
```

Upit za korisnike sada vraća sadržaj potpuno druge tablice:

```
1 rezultat (3 redaka):
2 (1, 'ana', '4111 1111 1111 1111')
3 (2, 'marko', '5500 0000 0000 0004')
4 (3, 'admin', '3400 0000 0000 009')
```

Brojevi kartica nisu ni bili dio prvotnog upita. Napadač ih je dohvatio kroz polje za prijavu.

4.3 Isti provjera protiv sigurne inačice

Sigurna funkcija predaje vrijednosti odvojeno od teksta upita, kroz upitnike:

```
1 def prijava_sigurna(veza, ime, lozinka):
2     upit = (
3         "SELECT id, korisnicko_ime, uloga FROM korisnici "
4         "WHERE korisnicko_ime = ? AND lozinka_hash = ?"
5     )
6     return veza.execute(upit, (ime, lozinka)).fetchall()
```

Listing 4.2: Sigurna funkcija za prijavu

Isti napadi protiv ove inačice ne rade. Ulaz admin' -- tretira se kao doslovno korisničko ime, koje ne postoji, pa prijava vraća prazan skup:

```
1 upit koji baza vidi:
2 SELECT ... WHERE korisnicko_ime = ? AND lozinka_hash = ?
3 parametri: ("admin' -- ", 'bilo sto')
4 rezultat (0 redaka):
5 prijava odbijena
```

Razlika nije u provjeri ulaza. Nijedan znak nije uklonjen ni promijenjen. Razlika je u tome što baza sada zna gdje prestaje naredba, a gdje počinje vrijednost.

5 | Zaštita

5.1 Pripremljeni upiti

Pripremljeni upit (engl. *prepared statement*) šalje tekst upita bazi odvojeno od vrijednosti. Baza prvo razumije strukturu naredbe, s praznim mjestima za vrijednosti, a tek onda dobiva podatke. Vrijednost više ne može promijeniti strukturu jer struktura je već određena.

Ovo je jedina mjera koja uklanja uzrok. Sve ostalo su dodatni slojevi. Listing 4.2 pokazuje oblik u jeziku Python, a isti pristup postoji u svakom ozbiljnom sučelju prema bazi. U jeziku PHP to je `mysqli::prepare` ili PDO s vezanjem parametara.

Jedno ograničenje treba znati. Kroz parametar se predaju vrijednosti, ne dijelovi sintakse. Ime tablice, ime stupca ili smjer sortiranja ne mogu ići kroz upitnik. Kad ti dijelovi ovise o korisniku, rješenje je bijela lista dopuštenih vrijednosti:

```
1 $dozvoljeni = ["asc" => "ASC", "desc" => "DESC"];  
2 $smjer = $dozvoljeni[strtolower($poredak)] ?? "ASC";
```

Ulaz se ovdje ne ubacuje u upit nego bira jednu od unaprijed poznatih vrijednosti. Nepoznat ulaz padne na sigurnu zadanu vrijednost.

5.2 Provjera ulaza

Provjera ulaza koristi, ali kao dopuna, ne kao zamjena. Ako polje treba sadržavati samo znamenke, odbacivanje svega ostalog smanjuje prostor za napad. Problem je u tome što ispravan ulaz za mnoga polja uključuje apostrof, primjerice prezime D'Angelo. Zato provjera ne može biti jedina obrana. Uz pripremljene upite takva imena rade bez ikakvog posebnog rukovanja.

5.3 Najmanje ovlasti

Račun baze kojim se aplikacija spaja treba imati samo ona prava koja mu trebaju. Aplikacija koja samo čita ne treba pravo pisanja ni brisanja. Ako napad ipak uspije, šteta je ograničena onim što taj račun smije. U demonstraciji sigurna inačica koristi račun `webapp_ro` s pravom čitanja, dok ranjiva koristi puni račun.

5.4 Slojevi iznad aplikacije

Web aplikacijski firewall (WAF) pregledava dolazni promet i blokira zahtjeve nalik napadu. Sustav za praćenje bilježi sumnjive upite. Ovi slojevi hvataju napade koje kod propusti i daju vrijeme za reakciju. Ne zamjenjuju ispravan kod jer rade s uzorcima, a napadač uzorke može zaobići. Korisni su kao dodatna crta obrane, ne kao prva.

Mjera	Uklanja uzrok	Uloga
Pripremljeni upiti	da	osnovna obrana
Bijela lista za sintaksu	da	za imena i sortiranje
Provjera ulaza	ne	smanjuje prostor napada
Najmanje ovlasti	ne	ograničava štetu
WAF i praćenje	ne	dodatni sloj

Tablica 5.1: Mjere zaštite i njihova uloga.

6 | Zaključak

SQL injection ostaje čest jer njegov uzrok djeluje bezazleno. Spajanje ulaza u tekst upita normalan je potez dok netko ne pokaže što se time otvara. Demonstracija u ovom radu prikazala je tri napada na istoj bazi, od zaobilaženja prijave do čitanja tablice koja nije bila dio upita, i sve kroz obično polje obrasca.

Obrana je poznata i nije skupa. Pripremljeni upiti razdvajaju naredbu od podatka i time uklanjaju uzrok, a ne samo pojedini simptom. Provjera ulaza, najmanje ovlasti i slojevi poput WAF-a dodaju sigurnost, ali ne zamjenjuju taj temelj. Aplikacija koja od početka koristi pripremljene upite zatvara cijelu ovu kategoriju napada, i to bez dodatnog troška u odnosu na ranjivu inačicu.

Literatura

- [1] OWASP FOUNDATION, *OWASP Top 10:2021*, dostupno na <https://owasp.org/Top10/>, pristupljeno u kolovozu 2026.
- [2] OWASP FOUNDATION, *SQL Injection Prevention Cheat Sheet*, dostupno na <https://cheatsheetseries.owasp.org/>, pristupljeno u kolovozu 2026.
- [3] J. CLARKE, *SQL Injection Attacks and Defense*, 2. izdanje, Syngress, Waltham, 2012.
- [4] MITRE, *CWE-89: Improper Neutralization of Special Elements used in an SQL Command*, dostupno na <https://cwe.mitre.org/data/definitions/89.html>, pristupljeno u kolovozu 2026.
- [5] SQLITE, *Query Language Documentation*, dostupno na <https://sqlite.org/lang.html>, pristupljeno u kolovozu 2026.

Sažetak

SQL injection je napad u kojem korisnički ulaz iz podatkovnog dijela upita prijeđe u njegov naredbeni dio. Rad opisuje mehanizam napada na razini jednog upita, razvrstava napade na one s izravnim odgovorom, slijepe napade i napade drugog reda, te kroz demonstraciju u kontroliranom okruženju pokazuje zaobilaženje prijave i čitanje tuđe tablice. Zaštita se gradi oko pripremljenih upita, koji razdvajaju naredbu od podatka i uklanjaju uzrok napada, uz dodatne slojeve poput provjere ulaza, najmanjih ovlasti i web aplikacijskog firewalla.

Ključne riječi

sql injection, sigurnost baza podataka, pripremljeni upiti, owasp, web sigurnost, napad ubacivanjem

Naslov rada na engleskom jeziku

Summary

SQL injection is an attack in which user input crosses from the data part of a query into its command part. This paper describes the mechanism at the level of a single query, groups attacks into in-band, blind, and second-order types, and demonstrates in a controlled environment how an attacker bypasses login and reads an unrelated table. Defense is built around prepared statements, which separate command from data and remove the root cause, together with supporting layers such as input validation, least privilege, and a web application firewall.

Keywords

sql injection, database security, prepared statements, owasp, web security, injection attack